

Visual-Inertial Odometry on a DJI Tello

Algorithms, ROS 2 architecture, calibration and measured results
for metric pose estimation without GPS

A stock Tello gives you a soft 720p stream 250 ms late, fused attitude in whole degrees at 10 Hz, and **no gyroscope at all**. Textbook tightly-coupled VIO assumes none of that is true. This report is about what you build instead, why, and how well it works.

Absolute trajectory error, 60 s / 40 m simulated flight	0.57 m RMS
Final drift as a fraction of path length	1.4 %
Metric scale accuracy (no GPS, no motion capture)	1.034
Roll / pitch RMS error	1.8°
Visual front-end cost	~3 ms / frame
Tests, all passing without a drone	116
Confirmed defects found and fixed in the existing code	30

Repository: [tello-vio](#) · ROS 2 Humble / Ubuntu 22.04 · packages [tello](#), [tello_msg](#), [tello_control](#), [tello_vio](#), [orb_slam2](#)

Contents

1	Executive summary What was found, what was built, what it achieves
2	The platform you actually have Why a Tello is not a normal VIO rig
3	Code review findings 30 confirmed defects, and the ones that mattered
4	Mathematical foundations Manifolds, error states, and why the convention is load-bearing
5	IMU preintegration Compressing a burst of inertial data into one constraint
6	The visual front-end Tracking, model selection, degeneracy, uncertainty
7	Fusion I — the error-state Kalman filter The real-time path, with stochastic cloning
8	Fusion II — the fixed-lag smoother Factor graphs and why marginalisation is not optional
9	Calibration Three quantities you must measure yourself
10	ROS 2 concepts, as used here Executors, QoS, TF, and the decisions they force
11	The pipeline end to end One image and one telemetry packet, followed all the way through
12	What went wrong in flight Three hypotheses, two refuted, one root cause
13	Results Measured performance, and what the camera is actually worth
14	Bring-up procedure The order to do things in
15	Limitations and next steps What this does not do
A	Reference Files, parameters, topics

1 Executive summary

What was found, what was built, and what it achieves

This report covers two pieces of work on the `tello-vio` repository: an adversarial review of the existing ROS 2 code, and the design and implementation of a visual-inertial odometry system on top of it.

The review

Thirty defects were confirmed against the real code, real ROS 2 semantics and real DJI SDK behaviour. Several were reachable in flight. The most serious:

- **Blocking SDK calls ran inside ROS callbacks.** `land()` retries for up to 21 s. On a single-threaded executor that froze /emergency *and* the RC dead-man for the entire window, so a drone commanded forward kept flying forward with no way to stop it.
- **The IMU topic published a fabricated zero angular velocity with a zero covariance** — which in ROS means “measured, and exactly known”. The Tello has no gyroscope; any consumer would have believed and integrated that zero.
- **Accelerations were published in the drone's FRD frame with a 2 % scale error**, so a level, stationary drone reported $z \approx -10 \text{ m/s}^2$ instead of $+9.81$.
- **camera_info was not rescaled when the image was downsampled.** At `video_scale=0.5` it claimed twice the true focal length — an error SLAM absorbs as scale and never recovers from.
- **The ORB-SLAM2 wrapper computed a camera pose every frame and threw it away.** No TF, no odometry. A SLAM node that publishes no pose cannot localise anything.
- **ORB_SLAM2_ENABLE=OFF forced the build instead of skipping it**, and `scripts/build.sh` combined a relative `cd` with a recursive delete.

All thirty are fixed. Section 3 gives the full list with failure scenarios.

What was built

A new `tello_vio` package, layered so that everything except the ROS nodes is pure NumPy/OpenCV and therefore testable and profilable without ROS installed:

- **Visual front-end** — grid-bucketed Shi-Tomasi detection, pyramidal KLT tracking with forward-backward validation, essential/homography model selection, and a rotation-compensated keyframe trigger. ~3 ms per frame.
- **Error-state Kalman filter** — 22 states including a stochastic clone, fusing attitude, the drone's own optical-flow velocity, barometer, ToF, zero-velocity updates and relative visual measurements. This is the real-time path.
- **Fixed-lag factor-graph smoother** — IMU preintegration, visual and unary factors, with proper Schur-complement marginalisation, verified to reproduce the full-batch solution.
- **Calibration** — Allan-variance IMU noise identification, rotation-only hand-eye for the camera-IMU extrinsic, and cross-correlation time-offset estimation, each reporting an observability score rather than just a residual.
- **ORB-SLAM2 backend** — now publishes pose, TF, path and tracking state, with a Sim(3) alignment node that turns loop closures into a `map→odom` correction so the odometry never jumps.

The one design decision everything follows from

A monocular camera cannot observe scale. On this airframe the metric anchor is the Tello's own downward optical-flow velocity, supported by the barometer and ToF. So the visual measurement is deliberately restricted to **rotation and translation direction** — a 2-degree-of-freedom bearing on the unit sphere — and never a magnitude. Section 7 derives it; Section 11 measures what happens when you take the flow sensor away.

What it achieves

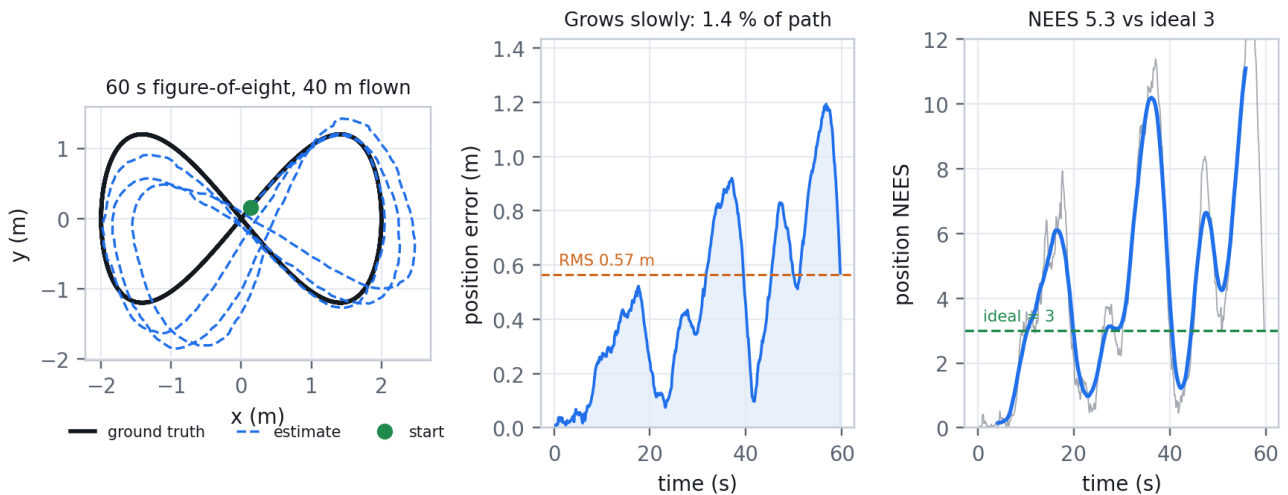


Figure 1 — 60 s simulated flight with the real drone's limitations modelled: 10 Hz jittery telemetry, whole-degree attitude, milli-g accelerometer quantisation, integer-centimetre velocity, a drifting barometer and a free-running yaw. Left: trajectory. Centre: position error. Right: covariance calibration.

2 The platform you actually have

Why a Tello is not a normal visual-inertial rig

Every design decision in this repository traces back to a handful of hardware facts. They are unglamorous and they are not negotiable, so they come first.

2.1 What the drone sends

The Tello exposes three UDP endpoints on its own WiFi access point. There is no ROS on the aircraft and no compute available to us on it: **everything in this report runs on the ground station**. “Computationally efficient” therefore means *fits inside a 33 ms frame budget on one laptop core*, not *fits on the drone*.

Port	Contents	Rate (measured)	What it costs you
11111	H.264 video, 960×720	27.6 Hz median (17.0 – 30.1)	150–350 ms latency, jittery, no hardware timestamp, visible compression artefacts
8890	State broadcast: attitude, accelerations, velocities, barometer, ToF, height, battery, temperatures	9.99 Hz median (9.49 – 10.53)	whole-degree attitude; milli-g accelerations; velocity in integer units; no angular rate
8889	Commands and responses	on demand	every command is a blocking round trip with a 7 s timeout and 3 retries

*Rates are medians over a 70-second flight log, computed from the estimator’s own message counters — not quoted from the datasheet. The documented figures (30 fps, 10 Hz) are close for telemetry and optimistic for video. **2.8 images arrive per telemetry packet**, and Section 11.4 shows how that ratio produced a real scheduling defect.*

2.2 There is no gyroscope

This is the fact that reshapes the whole project. The state packet carries *fused attitude* — pitch, roll, yaw as whole degrees — not angular rate. Every textbook tightly-coupled VIO system (VINS-Mono, OpenVINS, ORB-SLAM3-VI, MSCKF) preintegrates 100–200 Hz gyroscope and accelerometer data between camera frames. **That input does not exist on this aircraft.**

The only route to an angular rate is differentiating orientation, and doing it correctly means doing it on the manifold rather than subtracting Euler angles (which breaks at wrap-around and is simply wrong whenever roll and yaw both change):

$$\omega = \text{Log}(\mathbf{q}_{k-1}^{-1} \otimes \mathbf{q}_k) / dt$$

The resulting signal is real but weak. A uniform quantiser of width q has standard deviation $q/\sqrt{12}$; differentiated over dt that becomes $q/(dt\sqrt{6})$, which at one degree and 10 Hz is roughly **0.07 rad/s** $\approx$ **4°/s** of noise on a signal whose useful range is a few rad/s. It is good enough to predict where features will move and to gate outliers. It is not good enough to integrate.

Consequence: loosely coupled, with an explicit scale source

Tightly-coupled VIO is off the table, so this system is loosely coupled and gets its metric scale from sensors that are already metric. That is not a compromise forced by laziness — given this sensor suite it is the *correct* architecture, and Section 11 measures the cost of pretending otherwise. If you bolt a real IMU to the airframe, the preintegration in Section 5 is rate-agnostic and runs unchanged.

2.3 Units and frames are not ROS units and frames

The SDK reports in an aerospace FRD/NED convention: body axes x forward, y right, z down, with yaw increasing clockwise seen from above. ROS mandates FLU/ENU (REP-103). The body-frame conversion is a 180° rotation about x, $\text{diag}(1, -1, -1)$; for the attitude it negates pitch and yaw and leaves roll alone.

Field	Raw meaning	SI conversion	Trap
agx/agy/agz	specific force, milli-g, FRD	$\times 9.80665/1000$, then FRD→FLU	dividing by 100 gives 10.0 at rest — looks like 9.81 and is 2 % high
vgx/vgy/vgz	velocity from the onboard optical flow	$\times 0.01$ (centimetres/s), FRD→FLU	the SDK documents no unit here, though it says cm/s for every other speed. Assuming decimetres cost a 10× runaway in flight — Section 12
baro	pressure altitude in metres	relative use only, with an estimated bias	referenced to sea level, not to takeoff; it drifts with weather
tof	downward range, centimetres	$\div 100$, gated to 0.1–1.2 m	reads garbage over table edges and stairs
pitch/roll/yaw	degrees, NED-referenced	roll, -pitch, -yaw	yaw has no absolute reference — no usable magnetometer

Every one of these is a **parameter** rather than a hard-coded constant, because the SDK documentation is thin and firmware-dependent. That was the right structure and it was still not enough: `speed_to_mps` shipped at the wrong value, and because it sets the estimator's metric scale — the one quantity vision structurally cannot check — nothing detected it until the aircraft flew. Documenting an assumption is not the same as verifying it.

A level Tello at rest reports $\text{agz} \approx -1000$. An accelerometer measures specific force $f = a - g$; at rest in an FRD frame that is $[0, 0, -9.81]$, which is exactly the observed sign and confirms the frame reading. After conversion the same drone reports +9.81 on z, which is what REP-145 requires.

2.4 Timing is the quiet killer

Video and telemetry arrive on different sockets with different, unequal latencies, and neither is hardware-timestamped. At 1 rad/s of body rate, **50 ms of unmodelled camera-IMU offset injects 50 mrad — about 3° — of rotation error into every visual measurement**, which is an order of magnitude larger than the measurement noise itself.

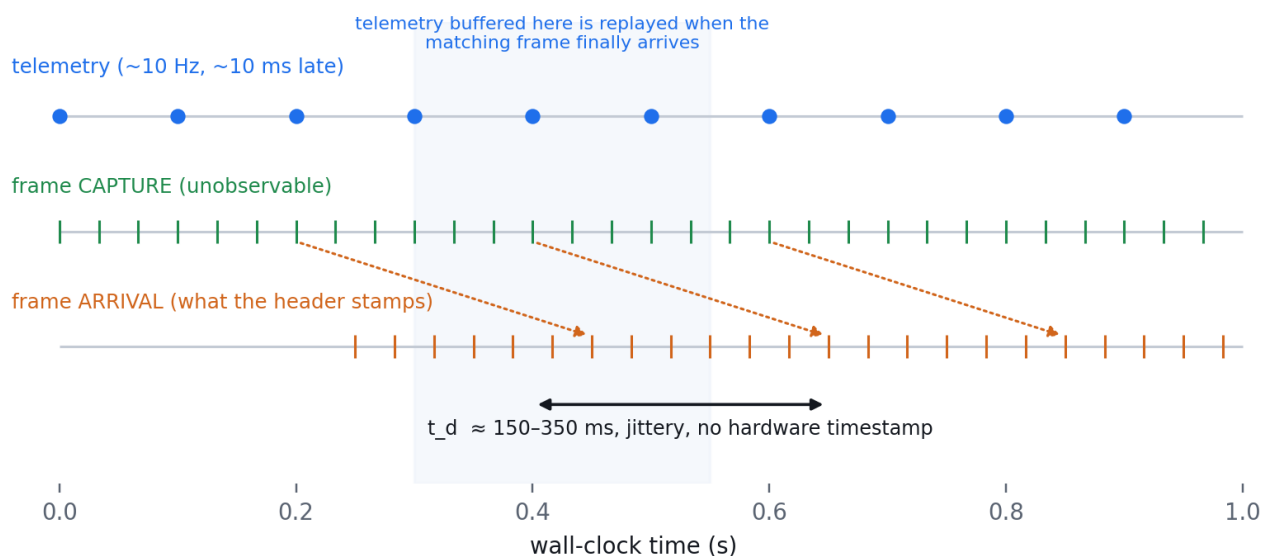


Figure 2 — The timing problem. Frames are stamped when they arrive, not when they were captured. The estimator therefore buffers telemetry and advances only to each frame's offset-corrected capture time, rather than fusing measurements in arrival order.

Two things follow. First, the offset t_d must be estimated (Section 9.3) and subtracted from every image stamp. Second, the estimator must run on the *image* clock: telemetry is buffered and replayed up to each frame's corrected time. The published odometry is consequently ~250 ms old but self-consistent; a separate topic forward-propagates to now for controllers, clearly labelled as a prediction and with an inflated covariance.

3 Code review findings

Thirty confirmed defects, ranked by what they do in flight

Findings were produced by five independent review passes — driver runtime, sensor physics, the SLAM wrapper, the build system, and VIO readiness — and each candidate was then handed to a separate adversarial verifier whose brief was to *refute* it against the real code, real ROS 2 semantics and real djitellopy behaviour. Ten claims were rejected that way, including some plausible ones: the `cb_control` axis mapping is non-standard but self-consistent end to end, and the `-00` build costs ~2.4 ms/frame rather than the catastrophe first claimed. Only what survived verification is listed here.

	Location	Defect	Effect
CRIT	tello/node.py	Blocking SDK commands in subscriber callbacks	<code>land()/takeoff()/flip()</code> block up to 21 s retrying. On a single-threaded executor /emergency and the RC dead-man are unavailable for that whole window, and the Tello latches the last rc setpoint — a drone told to fly forward keeps flying forward.
CRIT	tello/node.py	Three blocking query_* round trips inside the 2 Hz status timer	With <code>/status</code> and <code>/id</code> subscribed the executor blocked ~300 ms out of every 500 ms; on a degraded link a single tick stalled the node for up to 30 s.
CRIT	tello/node.py	/imu published angular_velocity = 0 with zero covariance	ROS semantics: all-zero covariance means 'measured and exactly known'. The Tello has no gyro. Any consumer would integrate a confident zero rate.
CRIT	tello/node.py	IMU acceleration published in the drone's FRD frame	Gravity pointed the wrong way: $z \approx -10 \text{ m/s}^2$ at rest instead of $+9.81$, with a 2 % scale error on top.
CRIT	tello/node.py	Attitude published without the NED→ENU conversion	Yaw and pitch ran backwards relative to REP-103.
CRIT	orb slam2/...node.cpp	Tcw computed every frame and discarded	No TF, no Odometry, no PoseStamped. The SLAM node localised nothing.
HIGH	tello/node.py	self._resize_cache read but never initialised	<code>AttributeError</code> killed the node for any <code>video_scale</code> not within $1e-6$ of 0.5 or 0.25.
HIGH	tello/node.py	camera_info not rescaled with video_scale	At 0.5 the published intrinsics claimed twice the true focal length; every derived bearing was wrong by a factor of two.
HIGH	tello/node.py	tello_ip and connect_timeout silently ignored	djitellopy binds the host in <code>__init__</code> and freezes <code>RESPONSE_TIMEOUT</code> as a default argument at import time, so assigning the class attributes did nothing.
HIGH	tello/node.py	Frames published as bgr8 but decoded as RGB	djitellopy returns <code>np.array(frame.to_image())</code> — RGB. Red and blue were swapped for every colour consumer.
HIGH	tello/node.py	No new-frame detection	The same frame was republished with a fresh timestamp, fabricating motion-free evidence and hiding a dead decoder behind a healthy-looking 30 Hz topic.
HIGH	tello/node.py	IMU timer-driven off a polled dict, not packet-driven	Timestamps were the timer's firing instant, displaced from the measurement by an unbounded 0–100 ms. A fixed offset can be calibrated out; a varying one cannot.

	Location	Defect	Effect
HIGH	tello/node.py	Odometry twist 10× too small, FRD axes, empty child_frame_id	robot_localization's transform lookup fails on an empty frame id and drops the measurement silently.
HIGH	tello/node.py	TF z from absolute barometric altitude	Published field elevation (~100 m) as height, on the same TF edge as a static publisher asserting zero — two owners, one edge.
HIGH	tello/node.py	Camera and IMU shared one frame_id	The camera-IMU extrinsic was literally inexpressible anywhere in the system.
HIGH	orbslam2/...node.cpp	Map points published in optical axes into 'map'	The map appeared on its side in RViz and was wrong for every consumer.
HIGH	orbslam2/...node.cpp	Whole map re-serialised into Markers every frame	O(map size) work on the tracking thread at 30 Hz.
HIGH	orbslam2/CMakeLists.txt	ORB_SLAM2_ENABLE=OFF forced the build	The find_package was inside the guard but add_executable was not, so the documented off switch caused a hard compile failure.
HIGH	orbslam2/CMakeLists.txt	config.yaml never installed	ORB-SLAM2's System constructor calls exit(-1) on a missing settings file, before any logging exists. README and launch both pointed at paths that do not exist.
HIGH	workspace/src/launch.py	Driver remapped /image_raw to /camera	Every consumer — tello_control, the SLAM wrapper, all ROS image tools — subscribes to /image_raw. The camera window stayed black.
HIGH	scripts/build.sh	Relative cd followed by rm -rf	'cd ../workspace && rm -rf build install log' deleted three directories relative to wherever the caller happened to be.
HIGH	scripts/install.sh	Installed ROS 2 Foxy into a Humble repository	Also used the deprecated apt-key mechanism.
MED	orbslam2/...node.cpp	annotated_frame stamped with wall-clock publish time	Could not be time-aligned with its own source image.
MED	FindORB_SLAM2.cmake	Accepted an include layout the sources cannot compile against	Configure succeeded and the compile then failed — a confusing place to find a search-path problem.
MED	tello_control/main.cpp	1 ms timer wrapping a blocking waitKey(15)	Spun a core at ~66 Hz to render a 30 Hz stream, and re-created the window every tick.
MED	tello_msg/TelloStatus.msg	No Header	No timestamp, so it could not be time-aligned or used with message_filters.
MED	tello/package.xml	Declared rosidl generators for a package with no messages	
MED	orbslam2/package.xml	Eigen, OpenCV and Boost find_package'd but not declared	rosdep reported success and the build then failed.
MED	orbslam2/CMakeLists.txt	CMAKE_BUILD_TYPE never defaulted	The wrapper compiled at -O0. Measured cost ~2.4 ms/frame — real, but far smaller than first assumed, since the heavy work is inside a Release-built libORB_SLAM2.so.
MED	tello/node.py	video_backend documented in the README but never declared	Passing it would have raised ParameterNotDeclaredException. Removed from the docs rather than faked.

The pattern worth noticing

Almost none of these produce an error message. A frozen executor looks like a laggy link; an unscaled camera_info looks like SLAM being bad at its job; a fabricated zero gyro reading looks like a drifting filter. Every one of them degrades gracefully into something you would misattribute to tuning. That is why the rewrite leans so heavily on making the invalid states *representable* — covariance[0] = -1 for unmeasured quantities, +inf for out-of-range ToF, None rather than zero for a missing angular rate.

4 Mathematical foundations

Manifolds, error states, and why the convention is load-bearing

4.1 Rotations do not live in a vector space

Position and velocity are vectors: adding a small correction to them is meaningful. Orientation is not. $SO(3)$ is a curved three-dimensional manifold with no global three-parameter chart, which is why every naive parameterisation fails somewhere — Euler angles at gimbal lock, a four-vector quaternion by carrying a singular covariance and needing constant re-normalisation.

The standard resolution, and the one used throughout this codebase, is to keep the *nominal* state on the manifold where it is free to be large, and to keep a *small* error in the tangent space where a Gaussian is actually a reasonable model. The two are connected by the exponential map:

```
Exp : R^3 -> SO(3)      Exp(phi) = I + sin|phi|/|phi| [phi]x
                        + (1-cos|phi|)/|phi|^2 [phi]x^2

Log : SO(3) -> R^3      the inverse, on (-pi, pi]
```

`tello_vio/lie.py` implements these with Taylor branches below 10^{-4} rad — where the closed forms lose precision to cancellation in $(1 - \cos \theta)/\theta^2$ — and a quaternion-based branch near $\theta = \pi$ where $\sin \theta \rightarrow 0$ makes the trace formula blow up. Both branches are exercised by tests.

4.2 The convention, stated once

Right-multiplicative, body-frame error

$R = R_{\square} \cdot \text{Exp}(\delta\theta)$ and $q = q_{\square} \otimes \text{Exp}_q(\delta\theta)$, with $\delta\theta$ in the body frame.

This is the choice made by Forster's preintegration and by GTSAM. The alternative — a left, world-frame error — yields *different* Jacobians for every single measurement. Mixing the two is the classic silent bug: the filter stays numerically well-behaved, the covariance still shrinks, and the estimate walks steadily away from the truth.

Because the convention is load-bearing, it is declared in exactly one place (`lie.py`'s module docstring) and every derived Jacobian is checked against a finite difference taken through the *same* retraction. That discipline is not decorative: it caught two genuine sign errors in the filter during development — the attitude update's H and the visual bearing residual — either of which would have made the filter correct in the wrong direction.

4.3 The right Jacobian

The one piece of Lie-group machinery that appears everywhere below is the right Jacobian, defined by how the exponential map responds to a perturbation of its argument:

```
Exp(phi + dphi) ~= Exp(phi) · Exp( Jr(phi) dphi )

Jr(phi) = I - (1-cos t)/t^2 [phi]x + (t - sin t)/t^3 [phi]x^2 ,    t = |phi|
```

It is what converts “a small change in a rotation vector” into “a small body-frame rotation”, and it appears in the covariance propagation of preintegration, in the bias Jacobians, and in every rotation residual's derivative. Where a residual is itself a rotation vector, its inverse Jr^{-1} appears instead.

4.4 Frames used throughout

Frame	Convention	Meaning
map	ENU, z up	Globally consistent, loop-closed. May jump.
odom	ENU, z up	VIO origin, fixed at initialisation. Continuous; drifts slowly.
base_link	FLU: x forward, y left, z up	The body / IMU frame. All inertial quantities live here.
camera_optical	z out of the lens, x right, y down	REP-103 optical frame. All image geometry lives here.

The camera-to-body rotation is the fixed $R_z(-90^\circ) R_x(-90^\circ)$, which maps camera +z (forward) to body +x, camera +x (right) to body -y, and camera +y (down) to body -z. The actual mount deviates from nominal by a few degrees of manufacturing tolerance, and a few degrees of extrinsic error looks exactly like scale drift in the output — hence Section 9.2.

5 IMU preintegration

Compressing a burst of inertial data into a single constraint

Preintegration answers one question: given a burst of IMU samples between two keyframes, what single constraint do they impose on the two keyframe states? Naively you would re-integrate the whole burst every time an optimiser touches either pose — ruinously expensive inside a Gauss-Newton loop, since the optimiser moves those poses on every iteration.

5.1 The trick

Preintegration instead compresses the burst *once*, into rotation, velocity and position increments expressed **in the frame of the first keyframe**. Because they are relative, they stay valid as the optimiser moves both poses around:

```
dR_ij = prod_k Exp( (w_k - b_g) dt )
dv_ij = sum_k dR_ik (a_k - b_a) dt
dp_ij = sum_k [ dv_ik dt + 1/2 dR_ik (a_k - b_a) dt^2 ]
```

These relate the two states through gravity and elapsed time:

```
R_j = R_i dR_ij
v_j = v_i + g dt_ij + R_i dv_ij
p_j = p_i + v_i dt_ij + 1/2 g dt_ij^2 + R_i dp_ij
```

Here a_k is **specific force**, not acceleration: a level, stationary IMU reads $[0, 0, +9.81]$, not zero. Conflating the two is a 1 g error, which is not subtle but is remarkably easy to introduce when converting frames.

5.2 The one thing it cannot absorb: bias

The increments above are computed at a fixed bias linearisation point. When the optimiser changes its bias estimate, re-integrating from scratch would defeat the purpose, so analytic bias Jacobians are carried alongside and used for a first-order correction:

```
dR~ = dR · Exp( ddR/db_g · db_g )
dv~ = dv + ddv/db_a · db_a + ddv/db_g · db_g
dp~ = dp + ddp/db_a · db_a + ddp/db_g · db_g
```

The ordering trap

The position Jacobians must be updated using the *previous* step's velocity Jacobians, before those are themselves updated. Get the order backwards and the correction is subtly wrong in a way that only shows up as slow convergence. The test suite pins this down by re-preintegrating from scratch at a perturbed bias and demanding the first-order prediction match to 10^{-8} .

5.3 Covariance, and why it is checked against Monte Carlo

The 9×9 covariance propagates as $\Sigma \leftarrow A \Sigma A^T + B Q_d B^T$. The discretisation constant is easy to get wrong: a continuous density σ acting over dt has discrete variance σ^2/dt once the dt inside B is accounted for. A missing or extra dt changes the answer by an order of magnitude — and is **completely invisible in the mean**. The estimate looks fine; only the consistency is destroyed.

So the implementation is validated by sampling the same noise through the same model 4000 times and comparing per-axis standard deviations to the analytic ones. They agree within 20 %, which at 4000 samples is a tight test.

On a Tello specifically

The angular rate fed into this is the 10 Hz attitude-difference surrogate from Section 2.2, not a gyroscope. The mathematics is rate-agnostic and remains exactly correct — but the *information* the constraint carries is proportionally weaker, which is why the estimator leans on body velocity and height for anything long-horizon. The same code runs unchanged on a real IMU.

5.4 Degenerate edges

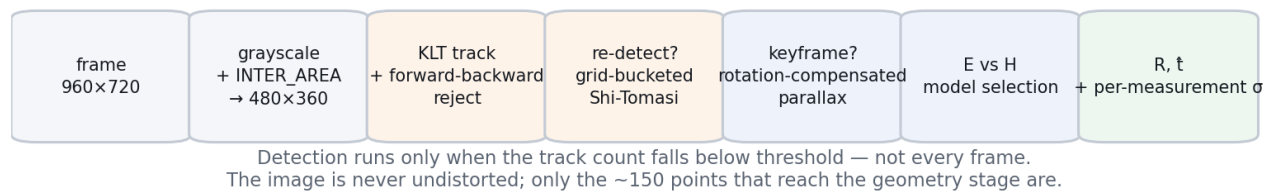
Two cases produce an information matrix that asserts near-infinite confidence, which would weld two keyframes rigidly together and dominate every other factor in the graph:

- **Empty increment.** The covariance is exactly zero, so its inverse is unbounded. Returns zero information — no constraint — rather than infinite.
- **Single sample.** Less obvious: the covariance is only rank 6, not 9. The noise mapping makes the position rows a fixed multiple of the velocity rows, so three directions carry no uncertainty at all and invert to infinity even though the matrix looks harmlessly non-zero. Handled by an eigenvalue cap, not just a diagonal floor — a floor merely converts “no information” into $1/\text{floor}$ of information.

6 The visual front-end

Tracking, model selection, degeneracy, and honest uncertainty

The front-end's job is to turn a soft, compressed, 250 ms-late video stream into one relative-pose measurement per keyframe, inside a 33 ms budget. It reports geometry only: rotation and translation *direction*, never a magnitude.



Measured: ~3 ms/frame at 480 px wide, ~150 features. ORB detect+describe+match is roughly 10× that.

Figure 3 — The per-frame pipeline.

6.1 Why KLT and not ORB every frame

ORB-SLAM2 detects and describes ~1000 ORB features on *every* frame and matches them by Hamming distance. That is the right architecture when the same features must also serve relocalisation and loop closure — which is exactly why ORB-SLAM2 remains the map backend in this project. It is the wrong architecture for a low-latency odometry front-end, where detect-and-describe dominates the per-frame cost.

	ORB detect+describe+match	Sparse KLT
Per-frame cost, 480×360, ~150 features	~25–30 ms	~3 ms
Sub-pixel accuracy between adjacent frames	quantised binary descriptor	direct photometric alignment
Survives total tracking loss	yes — descriptors allow relocalisation	no — this is the trade
Robust to large baselines	yes	no — needs small inter-frame motion

The weakness is real and is covered by design rather than denied: recovering from a total loss is ORB-SLAM2's job, running in parallel on the same stream.

6.2 Three details that decide whether it works

- **Forward-backward validation.** Track a point forward, then track the result back; if it does not land where it started, the track is wrong. One extra KLT call removes most of the drift-inducing bad matches that RANSAC would otherwise have to absorb.
- **Grid bucketing on detection.** `goodFeaturesToTrack` piles features onto the highest-contrast corner of the image — typically one window or one poster. Features clustered in a small image region give a nearly degenerate geometry and a translation direction with enormous variance. A per-cell quota spreads them out and conditions the whole estimate.
- **Never undistort the image.** Undistorting a full frame every cycle is pure waste. Distortion is a smooth warp that KLT handles happily; the correction is applied only to the ~150 *points* that reach the geometry stage.

6.3 Two-view geometry and its degeneracies

Given matched points, the relative pose comes from the essential matrix. Three things break this, and indoor flying hits all three routinely:

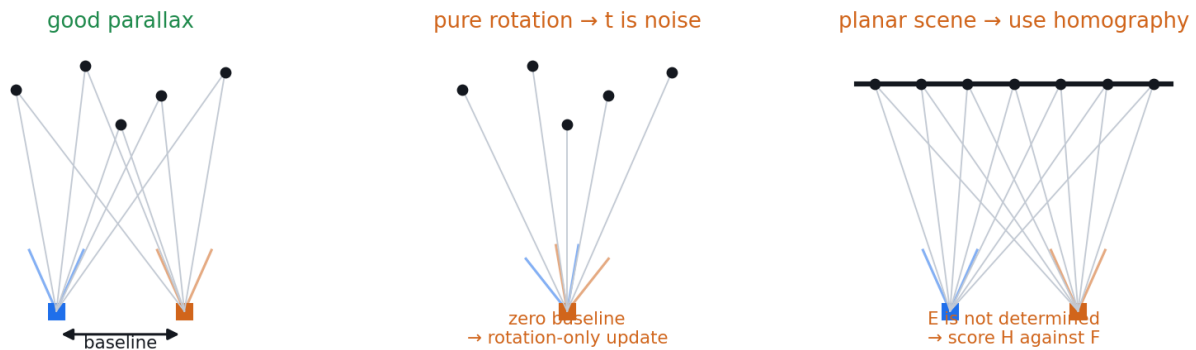


Figure 4 — The three configurations that break monocular two-view geometry.

- **Planar scenes.** Floors, walls and ceilings are the normal indoor case, and they are exactly where E is not uniquely determined and `recoverPose` returns confident nonsense. The front-end scores a homography against the fundamental matrix (Torr's symmetric-transfer approach, as used by ORB-SLAM) and switches models when the scene is plane-dominated.
- **Low parallax.** With little baseline the translation direction is unobservable — t is essentially a random unit vector. Parallax is measured *after de-rotating the bearings*, which is the only measure that separates “the drone rotated” from “the drone moved”; the raw bearing change conflates them completely.
- **Pure rotation.** The translation is genuinely zero. The front-end still returns the rotation — which is well conditioned and valuable — and flags the translation unreliable, so the filter applies a rotation-only update. Failing the whole measurement here would discard a good attitude constraint every time the pilot yaws on the spot, which on a Tello is most of the flight.

The keyframe trigger must measure parallax, not pixel motion

A rotation induces pixel motion identical in character to translation, so a raw-displacement trigger fires on *turning*, not on *moving* — the worst possible moment to make a keyframe, because the baseline is near zero.

Removing the prior rotation costs one 3×3 matrix and a projective divide over ~ 150 points. Measured effect on the rendered fly-through: median parallax at the keyframe rose from 0.6° to 1.5° , model selection stopped drifting to the homography, and rotation error fell from $2\text{--}4^\circ$ to $0.2\text{--}1.7^\circ$.

The prior comes free from the drone's attitude stream — the one piece of inertial data a Tello provides in good shape.

6.4 Honest per-measurement uncertainty

A fixed R matrix is wrong here. A 170-inlier essential solve at 3° of parallax and a 30-inlier homography at 0.3° are not the same measurement, and giving both the same confidence lets the bad ones dominate. Each measurement therefore reports its own σ :

```
sigma_rot = 0.033 rad · sqrt(60/N) · (3.5 if homography else 1)
sigma_dir = 0.128 rad · sqrt(60/N) · max(1, 3 deg / parallax_deg)
```

The $1/\sqrt{N}$ is averaging over correspondences; the $1/\text{parallax}$ is the geometry of a long thin triangle, which degrades fast as the baseline shrinks. **The constants are fitted, not invented:** regressing observed errors from 64 keyframes across 8 rendered sequences onto that shape gives 1.87° and 7.3° , at which the normalised errors have a 90th percentile of $1.4\text{--}1.7 \sigma$ and only $\sim 2 \%$ beyond 3σ . Those figures come from synthetic imagery with 0.3 px feature noise; a real Tello feed is worse, so they are a floor and both are node parameters.

7 Fusion I — the error-state Kalman filter

The real-time path, and where metric scale comes from

The ESKF is what runs in flight. It is cheap, it is causal, and it produces a pose at every telemetry epoch. Its cost, relative to the smoother in Section 8, is that it linearises once at the current estimate and never revisits that choice.

7.1 State layout

Index	Error state	Nominal quantity	Why it is there
0:3	δp	position of body in world (m)	
3:6	δv	velocity of body in world (m/s)	
6:9	$\delta \theta$	attitude q_{WB}	body-frame, right-multiplicative
9:12	δb_a	accelerometer bias (m/s ²)	observable through gravity and ZUPT
12:15	δb_ω	surrogate-gyro bias (rad/s)	observable through ZARU
15	δb_p	barometer bias (m)	the barometer is sea-level referenced and drifts
16:19	δp_c	cloned position at the last keyframe	stochastic cloning
19:22	$\delta \theta_c$	cloned attitude at the last keyframe	stochastic cloning

Twenty-two states. A 22×22 update is a few microseconds of NumPy, so the filter is nowhere near the compute budget — the front-end is.

7.2 Why there is a clone

Monocular vision naturally produces *relative* measurements: “between keyframe A and now, the camera rotated by this much and moved in this direction”. Feeding a relative measurement to a filter that only knows the current state is statistically wrong — the measurement is correlated with the past state error, and ignoring that correlation makes the filter over-confident and eventually divergent.

Stochastic cloning (Roumeliotis & Burdick, 2002) fixes it by carrying a copy of the state at the reference time inside the same covariance, so the cross-correlation is tracked explicitly. Cloning is not a copy of the mean: the clone inherits the full covariance *and* cross-covariance via $P \leftarrow J P J^T$.

A consequence that looks like a bug and is not

Immediately after cloning, the live state and the clone are perfectly correlated, so the filter knows the relative pose *exactly* — it is zero — and correctly ignores any relative measurement. Only accumulated process noise makes the baseline uncertain and therefore correctable. A test asserts precisely this, because the first instinct on seeing “the visual update did nothing” is to go hunting for a sign error.

7.3 The measurements, and what each one buys

Update	Model	What it makes observable
Attitude (AHRS)	$r = \text{Log}(q_m^{-1} \otimes q_m)$	Roll and pitch absolutely (gravity-referenced). Yaw only as a smoothness hint — no magnetometer, so a large yaw variance is used, built in <i>world</i> axes and rotated into the body so it stays attached to world-z under tilt.

Update	Model	What it makes observable
Body velocity (optical flow)	$h = R^T v$	Metric scale. The $\text{skew}(R^T v)$ term in the Jacobian is what couples a velocity residual into an attitude correction — the mechanism by which sustained horizontal flight observes roll/pitch bias.
Barometer	$h = p_z + b_p$	Vertical scale, once the bias is anchored at initialisation. Without that anchor p_z and b_p are jointly unobservable and drift together forever while the residual sits happily at zero.
ToF	$h = (p_z - z_g) / R[2,2]$	Absolute height below ~1.2 m. The tilt division is not optional: skipping it injects 6 % of height error at 20° of bank, an ordinary Tello manoeuvre.
ZUPT / ZARU	$v = 0, \omega_m = b_\omega$	The highest-information measurements available. While landed these are <i>exact</i> , turning bias from weakly to strongly observable in seconds.
Visual relative	rotation (3) + bearing (2) against the clone	Rotation and translation <i>direction</i> . Deliberately never magnitude.

7.4 The bearing residual

A bearing carries exactly two degrees of freedom, so its residual must live in the two-dimensional tangent plane of the unit sphere at the measured direction:

$$\begin{aligned} u_{\text{pred}} &= d(x) / |d(x)| & d(x) &= R_{BC}^T (w + (A - I) p_{BC}) \\ r_{\text{dir}} &= B^T (t_{\text{meas}} - u_{\text{pred}}) & B &= 3 \times 2 \text{ basis perpendicular to } t_{\text{meas}} \end{aligned}$$

Why not just use a 3-vector residual

Because the radial direction carries no information. Treating the bearing as a 3-vector residual — the common shortcut — silently adds a phantom constraint along that direction, which pulls on the translation *magnitude* and therefore biases the very scale that the flow sensor, barometer and ToF worked to establish. The 2-DoF residual is what keeps the split in Section 1 honest.

7.5 Numerical hygiene

- **Joseph-form update** — $(I - KH)P(I - KH)^T + KRK^T$. It stays symmetric positive-definite under round-off, which the short form does not once you run for minutes at 30 Hz with states scaled from metres to milliradians. One extra 22×22 product: free.
- **Chi-square innovation gating** at $3 \times$ the 95 % quantile — loose enough not to reject honest measurements during fast manoeuvres, tight enough to catch a mis-associated visual match or a ToF reading off a table edge.
- **Covariance reset on injection** — after folding $\delta\theta$ into the nominal state, the *meaning* of the remaining attitude error changes. Two lines, a second-order effect, and it keeps the filter consistent during aggressive rotation.
- **Bias clamping** — biases are physically bounded, so clamping stops a temporarily unobservable direction from ratcheting off during a long stationary segment.
- **Missing data is missing** — when the surrogate cannot produce a rate (dropout, duplicate packet) the filter propagates with no rotation and inflates the covariance. Substituting zero would assert “the drone did not rotate”, a confident lie.

8 Fusion II — the fixed-lag smoother

Factor graphs, and why marginalisation is not optional

A filter linearises once, at the current estimate, and lives with it. An early linearisation error — made when the attitude or scale was still poorly known — is baked in forever. A smoother keeps the last N keyframes as free variables and re-solves the whole window as new data arrives, so an early mistake gets corrected once later measurements disambiguate it. That is why VINS-Mono, OKVIS and ORB-SLAM3 are all optimisation-based.

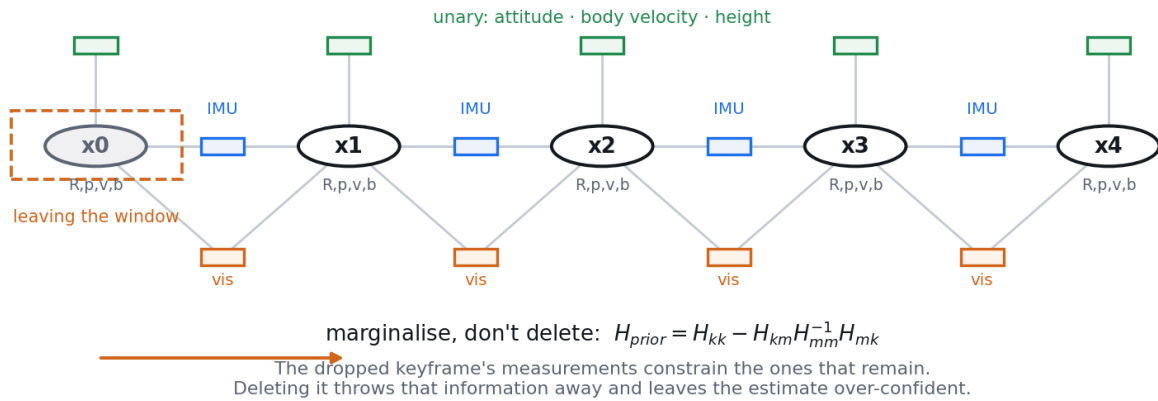


Figure 5 — The sliding window. Circles are keyframe states (R, p, v, bias); squares are factors. When a keyframe leaves the window its information is pushed into a dense prior on its neighbours rather than discarded.

8.1 The problem being solved

Maximum a posteriori over the window: minimise the sum of squared Mahalanobis residuals, on the manifold, by Levenberg-Marquardt with a Huber kernel on the visual factors.

```
min_x  sum_i  rho( || r_i(x) ||^2_{Lambda_i} )

solve  (H + lambda diag H) delta = b      with H = sum J^T Lambda J
then   x <- x [+] delta                    (retraction, not addition)
```

Each keyframe contributes 15 error dimensions $[\delta\theta, \delta p, \delta v, \delta b_a, \delta b_\omega]$. At the default window of 10 that is a 150×150 dense Cholesky — a few hundred microseconds, comfortably inside a keyframe interval. Past roughly $N = 25$ a sparse solver starts to pay for itself, and past that you want incremental factorisation (iSAM2) instead of re-solving from scratch.

A trap worth naming

Preintegration orders its state blocks $[\delta\theta, \delta v, \delta p]$ (following Forster); the smoother orders them $[\delta\theta, \delta p, \delta v, \dots]$. The IMU factor remaps between them explicitly rather than relying on the two happening to agree. Swapping p and v silently produces a graph that converges to the wrong trajectory.

8.2 Marginalisation is the whole point

When a keyframe leaves the window you cannot simply delete it. The measurements that touched it carry information about the keyframes that *remain*; dropping them throws that information away, which makes the

estimate both worse and over-confident about being worse. The correct operation is a Schur complement:

```
H_prior = H_kk - H_km H_mm^-1 H_mk
b_prior = b_k - H_km H_mm^-1 b_m
```

Three implementation details are load-bearing. **Only factors that touch the departing keyframe are eliminated** — folding untouched factors into the prior double-counts them on the next solve. **The prior is held at a fixed linearisation point** (a first-estimates-Jacobian convention); re-linearising it around a moving estimate injects information that was never measured and is the standard route to an inconsistent estimator. And **a pseudo-inverse is used for H_{mm}** , so a partly-observed departing keyframe marginalises its observed subspace and leaves the rest alone.

Verified, not asserted

The property that matters is **batch equivalence**: solving the window *after* marginalisation must give the same answer as solving the full batch. The test solves to convergence, marginalises at the optimum, kicks the survivors, re-solves, and demands they return to within 2 mm of the batch optimum.

A companion test quantifies what it buys: naive deletion of the same keyframe leaves an error more than 5× larger. A sliding window that deletes instead of marginalising still runs, still converges, and still looks plausible on a plot — it is just quietly wrong. Only this test catches that.

8.3 Observability, stated as a rank test

A visual-inertial graph measures relative geometry plus gravity. Gravity pins roll and pitch; nothing pins absolute position. So the Hessian of a window with no absolute-position measurement is rank deficient by exactly three, and the null space is “translate every keyframe identically”. The test suite asserts both the dimension and the direction of that null space, which documents why the position anchor exists and would catch a future factor that accidentally constrained something it cannot observe.

8.4 Which one should you run?

	ESKF (Section 7)	Fixed-lag smoother
Cost per update	~ μ s (22×22)	~100s of μ s (150×150), per keyframe
Linearisation	once, at the current estimate	repeatedly, over the window
Recovers from a bad early estimate	no	yes
Output rate	every telemetry epoch	every keyframe
Use it for	flight, control, the default path	post-flight refinement, bag replay, initialisation, and as ground truth when tuning the filter

Both share the same measurement models and the same conventions, so a discrepancy between them is a real signal that something is wrong — which makes the smoother useful as a cross-check even when it is not in the flight path.

9 Calibration

Three quantities you must measure on your own airframe

None of these is published by the manufacturer, and none can be guessed. Skipping any of them produces a confident wrong answer rather than an obviously wrong one, which is considerably more expensive.

9.1 IMU noise and biases

These set the process noise Q . Guess them too small and the filter over-trusts dead reckoning and diverges; too large and it ignores the IMU entirely. The tool is the **overlapping Allan deviation** of a static log, which separates white noise from bias instability rather than lumping them:

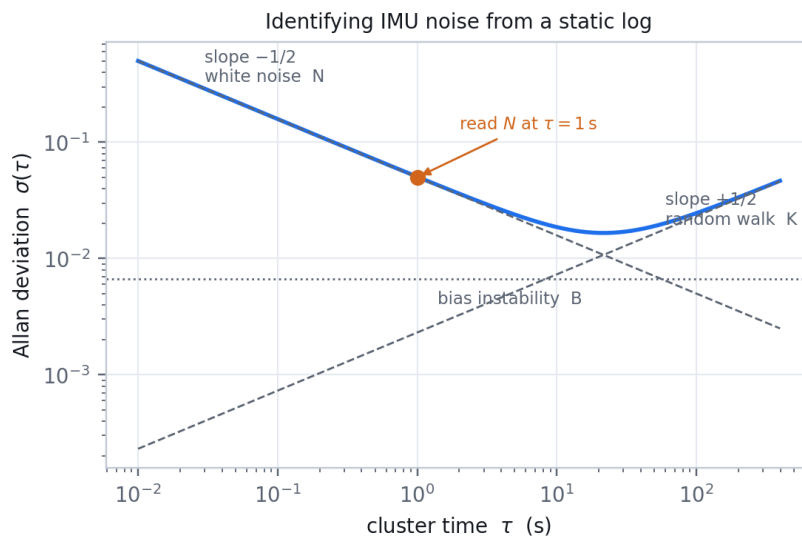


Figure 6 — Reading an Allan deviation plot. The $-1/2$ slope gives the white-noise density N at $\tau = 1$ s; the flat minimum is bias instability; the $+1/2$ tail gives the random walk K .

- **Two minutes, not ten seconds.** The estimator at cluster time τ averages over $N = 2\tau/dt$ overlapping windows, so the bias-instability and random-walk regions — the ones that set how long the filter may trust dead reckoning — only become visible once the log is many times longer than the τ you care about.
- **The drone must be genuinely still**, on a surface that is not transmitting vibration. Real motion inflates the apparent noise, which quietly makes the filter sluggish forever after. The node reports detected non-stationarity as a warning rather than folding it into the answer.
- **The worst axis is used, not the mean.** The filter's noise model is isotropic, and being optimistic on the noisiest axis is what makes it diverge.
- **Sanity check built in.** If the mean acceleration magnitude is not ~ 9.81 m/s², the node says so — raw milli-g fed in by mistake is the common cause.

9.2 The camera-IMU rotation, from motion alone

Every visual measurement is in the camera frame and every inertial one in the body frame; without the rotation between them they cannot be combined at all. **A 5° error here couples rotation directly into apparent translation and produces a drift that looks exactly like scale error** — which is why people chase the wrong bug for days.

No calibration target is needed. For a camera rigidly attached to a body:

$$R_{C1C2} = R_{CB} \ R_{B1B2} \ R_{BC}$$

Conjugation preserves the rotation ANGLE and maps the AXIS, so with n_C , n_B the unit rotation axes of the two measured increments:

$$n_C = R_{CB} \ n_B \quad \text{and} \quad |\theta_C| = |\theta_B|$$

That reduces hand-eye to Wahba's problem — find the rotation best aligning two sets of unit vectors — solved in closed form by SVD (Kabsch). No iteration, no initial guess, no local minimum.

Two screens, because one is not enough

Increments below a few degrees have noise-dominated axes and are dropped. Pairs whose two *angles* disagree violate the identity outright — almost always a time-sync error — and are dropped too.

That cheap screen is **not sufficient**: a corrupted axis that happens to preserve the rotation angle sails straight through it and biases the closed-form fit by degrees. A second stage re-fits after rejecting pairs whose full conjugation residual exceeds a median+3·MAD cut. The test that exercises this deliberately includes a blunder with 0.1° of angle mismatch, which only the second stage removes.

Excitation is reported alongside the residual. Rotating about a single axis leaves R_{BC} undetermined about that axis; the fit will still converge and still report a small residual. The node computes the smallest singular value of the stacked rotation axes and says POOR below 0.25. A small residual with a low excitation score means the answer is confidently wrong.

The translation p_{BC} is *not* fitted. It needs metric scale and well-excited translation; on a Tello the lever arm is a few centimetres and is far better taken from the mechanical layout than from noisy data.

9.3 The camera-IMU time offset

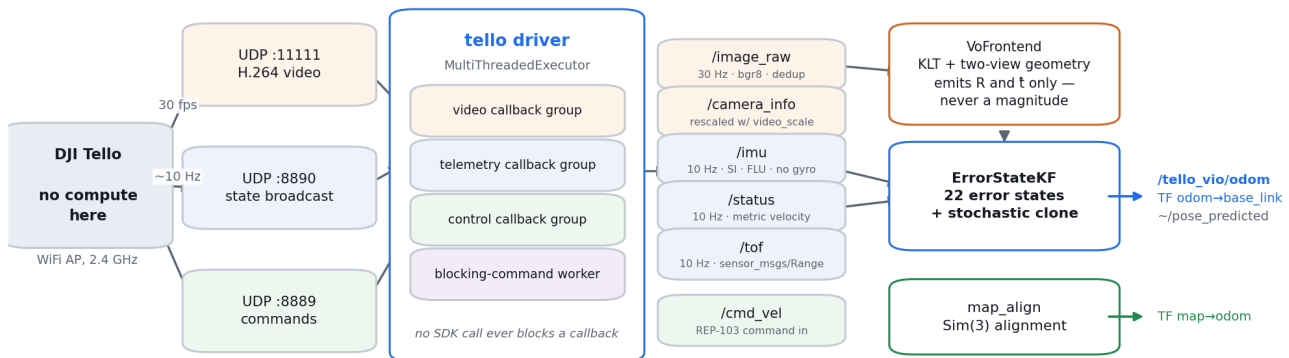
Recovered by cross-correlating angular-rate *magnitudes* — one from the visual front-end, one from the attitude surrogate. Magnitude is the right choice because it is frame-independent: it does not require R_{BC} to be known yet, so the offset and the extrinsic can be solved in either order. In practice the offset is solved first and applied before the hand-eye fit, because an uncompensated offset makes the paired rotations disagree and would otherwise be absorbed into a wrong extrinsic.

The correlation peak is refined to sub-sample resolution by fitting a parabola to the three values around the maximum. Without that the answer is quantised to the resampling period — 20 ms at the default — which is the same order as the effect being measured. The reported correlation coefficient is the quality metric: below ~0.5, excite the drone more.

10 ROS 2 concepts, as used here

Executors, callback groups, QoS, TF — and the decisions they force

ROS 2 concepts are covered here in the order they actually bite on this project, with the concrete decision each one forced.



Everything to the right of the drone runs off-board. The Tello streams; it does not compute.

Figure 7 — Node and topic graph.

10.1 Executors and callback groups

A ROS 2 node does not run its callbacks; an **executor** does. The default `SingleThreadedExecutor` runs one callback at a time, so a callback that blocks blocks *everything* — which is exactly how the original driver could freeze /emergency for 21 seconds.

Callback groups declare what may run concurrently. `MutuallyExclusiveCallbackGroup` guarantees its members never run simultaneously with each other; `ReentrantCallbackGroup` allows even a single callback to re-enter itself. Groups only take effect under a `MultiThreadedExecutor`.

Node	Groups	Why
tello (driver)	video · telemetry · control, plus a non-ROS command worker thread	Video decode must not delay telemetry ingest, and neither may delay the RC dead-man. Blocking SDK round trips are pushed off the executor entirely — a worker thread is the right answer because the Tello's command socket is single-threaded by protocol.
tello_vio	vision · telemetry	The image callback runs feature detection and RANSAC — tens of milliseconds. Telemetry must not queue behind it. Mutual exclusivity <i>within</i> each group means no callback races itself; one lock guards the estimator, the only shared mutable state.

The rule of thumb

Threads are not the fix for a blocking call — they are the fix for *concurrency*. A blocking call belongs off the executor entirely. Callback groups then let the genuinely independent work overlap.

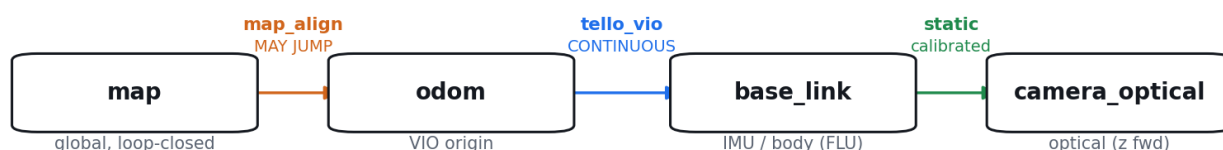
10.2 QoS: the silent no-connection

ROS 2 QoS profiles must be *compatible* for a subscription to connect at all, and an incompatible pair produces **no data and no error** — the most confusing failure mode in the middleware. Sensor streams here use

qos_profile_sensor_data (best-effort, volatile, depth 1) on both sides.

Setting	Choice	Reason
Reliability	BEST_EFFORT for image/IMU; RELIABLE for commands	Retransmitting a 250 ms-old video frame is worse than dropping it. Losing a takeoff command is not acceptable.
History / depth	KEEP_LAST, depth 1 for sensors	A deep queue on a latency-critical stream buys you stale data, not resilience.
Durability	VOLATILE for sensors; TRANSIENT_LOCAL for static TF	A node that starts late still needs the static transform; it does not need old frames.

10.3 TF and REP-105



REP-105: controllers differentiate odom→base_link, so it must never jump.
A loop closure is published as a correction to map→odom, which nothing differentiates.

Exactly one node owns each edge.

Figure 8 — The transform tree, and who owns each edge.

REP-105 defines the `map → odom → base_link` chain and, crucially, the different guarantees each edge carries. `odom → base_link` must be **continuous** because controllers differentiate it — a discontinuity there is a step input to the aircraft. `map → odom` is **allowed to jump** because nothing differentiates it.

That is what makes loop closure safe: when ORB-SLAM2 closes a loop and its trajectory snaps, the correction is published as a change to `map → odom`, and the VIO's `odom → base_link` is untouched. Publishing the loop closure straight into the odometry — the obvious-looking shortcut — is what makes drones lurch when a loop closes.

One owner per edge

The original launch files ran a static publisher on `map → drone` while the driver published a barometer-derived transform on the *same* edge. TF does not merge those: lookups return whichever sample won the buffer, and RViz shows the drone flickering between them. Every edge in this system has exactly one publisher, and the launch files enforce it.

10.4 Message design

- **Covariance conventions are semantics, not decoration.** `covariance[0] = -1` means “not measured”. All-zeros means “measured, and exactly known”. The driver now uses the former for the absent gyro and for the absent position, because the difference is what a downstream filter acts on.
- **child_frame_id on Odometry is required.** The twist is expressed in it. Leaving it empty makes `robot_localization`'s transform lookup fail and the measurement is dropped silently.
- **Every message needs a Header.** `TelloStatus` had none, so it could not be time-aligned with `/imu` or used with `message_filters`. Adding one was a prerequisite for fusing it.

- **Use the idiomatic type.** The downward rangefinder is now `sensor_msgs/Range`, which has a defined convention for out-of-range readings ($\pm\text{inf}$) — so “no reading” is representable instead of being encoded as a plausible number.

10.5 Parameters, launch and bags

- **Everything measured is a parameter.** Sensor unit scales, noise densities, the extrinsic, the time offset. The defaults are documented with their provenance and flagged `CALIBRATE` in the config where they must be replaced.
- **Launch arguments over edited files.** One launch file covers bench testing, flight and bag replay.
- **use_sim_time is wired through.** Without it, nodes stamp with wall time while a bag replays recorded stamps, and every time-based computation in the estimator silently becomes nonsense.
- **Record bags and iterate offline.** Recording `/image_raw /camera_info /imu /status /tof` lets you re-run calibration and re-tune the filter without flying — which matters more than usual here, because the Tello overheats sitting still and gives you ~10 minutes of battery.

```
ros2 bag record -o flight1 /image_raw /camera_info /imu /status /tof
ros2 bag play flight1 --clock
ros2 launch tello_vio vio.launch.py driver:=false use_sim_time:=true
```

10.6 Diagnostics

The estimator publishes `diagnostic_msgs/DiagnosticArray` at 2 Hz with per-measurement acceptance rates and the last normalised innovation squared, current bias estimates, per-axis sigmas, front-end status and the last visual measurement's model, inlier count and parallax. Watch it in `rqt_robot_monitor`. Falling visual acceptance is the earliest visible sign of a bad extrinsic, a wrong time offset, or a scene the front-end cannot track — all of which otherwise present identically as “drift”.

11 The pipeline end to end

One image and one telemetry packet, followed all the way through

Everything before this section described components. This one follows real data through them in order, naming the function that touches it at each step and the reason that step exists. Every rate quoted here was **measured on the aircraft**, not taken from a datasheet: see Section 2.2 for the measurement.

11.1 What arrives, and how often

Stream	Rate (measured)	Latency	Carried by
Video, UDP :11111	27.6 Hz median (17.0 – 30.1)	150 – 350 ms, jittery	H.264, 960×720, decoded by djitellopy
State, UDP :8890	9.99 Hz median (9.49 – 10.53)	~10 ms	one ASCII line, ~150 bytes
Commands, UDP :8889	on demand	blocking, 7 s timeout	text, one at a time

*The ratio matters more than either number: **2.8 images arrive per telemetry packet**. That single fact drives the scheduling design in 11.4 and caused a real defect described there.*

11.2 Telemetry path, packet by packet

The SDK broadcasts state; it is not polled. But djitellopy exposes only a dictionary that its receiver thread overwrites in place, with no sequence number and no arrival timestamp. So the driver polls that dictionary at 50 Hz and publishes only when the contents *change* (`_on_telemetry_timer`).

Polling faster than the source and detecting change is not a workaround for laziness — it is what bounds the timestamp error. Sampling on an independent 10 Hz timer beats one aperiodic stream against another: you get duplicated samples, dropped samples, and a stamp that is the *timer's* firing instant rather than the packet's arrival, displaced by an unbounded 0–100 ms. A fixed camera–IMU offset can be calibrated out; a randomly varying one cannot. Polling at 50 Hz bounds that displacement to one 20 ms poll period.

```
raw SDK line  pitch:0;roll:-1;yaw:35;vgx:42;vgy:0;vgz:-3;templ:62;temph:65;
               tof:104;h:100;bat:73;baro:105.30;time:0;agx:-3.00;agy:8.00;agz:-1002.00
```

conversions applied by the driver (tello/node.py):

```
agx,agy,agz  milli-g      -> m/s^2   x G0/1000, then FRD -> FLU
vgx,vgy,vgz  cm/s        -> m/s      x 0.01,   then FRD -> FLU
pitch,roll,yaw deg, NED-referenced -> quaternion, ENU-referenced
baro         (djitellopy x100) -> m      /100, biased, NOT height AGL
tof          cm           -> m      /100, valid 0.1-1.2 m only
h            cm           -> m      /100, decimetre resolution
```

*A level, stationary Tello reports $agz \approx -1002$. An accelerometer measures specific force $\mathbf{f} = \mathbf{a} - \mathbf{g}$; at rest $\mathbf{a} = 0$, so in an FRD frame (z down) $\mathbf{f} = [0, 0, -9.81]$. That is why the raw sign is negative, and why after the FLU conversion the published value is **+9.81** on z — which is what REP-145 requires a sensor_msgs/Imu publisher to emit.*

11.3 Manufacturing an angular rate that does not exist

The propagation model needs ω . The drone publishes orientation. Differentiating orientation is the only route, and doing it naively — subtracting Euler angles — breaks at wrap-around and is wrong whenever roll and yaw both change. The correct finite difference is taken on the manifold (`TelloImuSurrogate.update`):

$$\omega \approx \text{Log}(q_{\{k-1\}}^{-1} \otimes q_k) / dt$$

exact for a constant-rate rotation over the interval, with no wrap-around pathology. Two properties of the result must be respected downstream. First, its noise is dominated by *quantisation*, not by any MEMS gyro: whole-degree attitude at 10 Hz gives a uniform quantiser of width $q = 1^\circ$, whose standard deviation is $q/\sqrt{12}$, and differentiating over dt inflates that to

$$\sigma_\omega \approx q / (dt \cdot \sqrt{6}) \approx 1^\circ / (0.1 \text{ s} \cdot 2.449) \approx 4.1^\circ/\text{s} \approx 0.071 \text{ rad/s}$$

which is the value `gyro_noise_density` is set from. Second, when a packet is duplicated or a gap is too long, the surrogate returns **None** rather than a number, and the filter propagates on its bias estimate with inflated covariance. Substituting zero would assert “*the drone did not rotate*” — a confident lie that corrupts the attitude covariance and, through it, everything conditioned on attitude.

11.4 Scheduling: whose clock does the filter run on?

Video lands 150–350 ms after capture; telemetry in about 10 ms. Fusing each as it arrives would apply visual measurements to a state that has already moved on — an out-of-sequence measurement, which a plain EKF handles by being wrong. So telemetry is *buffered* (3 s ring) and the filter is advanced only up to each image’s offset-corrected capture time. The published odometry is therefore deliberately ~250 ms old but self-consistent; `~/pose_predicted` carries a forward extrapolation for control, on its own topic and with inflated covariance, so it cannot be mistaken for a measurement.

A defect this design created, and how it was found

Because images outnumber telemetry packets 2.8:1, an image time almost never coincides with a telemetry sample. `_advance_to` replayed buffered samples and stopped at the last one at or before the image time — it never propagated the remaining interval. Measured over the real rates, the state was a **mean of 49 ms and up to 99 ms behind** the image it was about to be compared against: 2.5 cm of un-modelled motion at 0.5 m/s injected into every visual update, and a published pose carrying a stamp it was not valid at.

Fixed by propagating the residual interval with a zero-order hold on the last IMU sample — what the propagation model already assumes between samples. The target is *clamped to the newest buffered telemetry*, because running the filter ahead of the telemetry clock would make the next arriving sample older than `last_fused_t` and get it silently dropped. `test_fusion_timing.py` pins both halves.

11.5 One image, step by step

#	Step	Where	Cost
1	Convert to grayscale, downscale to 480 px wide, rescale K to match (not D — distortion is dimensionless)	<code>VoFrontend._prepare</code>	0.3 ms
2	Track existing features into the new frame with pyramidal KLT, then track the result <i>back</i> and reject any point that does not return	<code>FeatureTracker.track</code>	0.8 ms
3	If fewer than 90 survive, top up with grid-bucketed Shi–Tomasi, one quota per cell	<code>FeatureTracker.detect</code>	1.9 ms (occasional)
4	Pair keyframe and live observations by feature id , never by array position	<code>matched_pairs</code>	<0.1 ms
5	Remove the predicted rotation from the flow and take the median: this is parallax, and it decides whether to make a keyframe	<code>rotation_compensated_flow</code>	<0.1 ms
6	<i>Only on a keyframe</i> : undistort the ~150 points, score H against F, recover (R, t), measure de-rotated parallax	<code>estimate_relative_pose</code>	1.0 ms
7	Attach a per-measurement σ from inlier count and parallax	<code>VisualMeasurement.noise_std</code>	<0.1 ms

11.6 Why each of those choices, and not the obvious alternative

KLT tracking rather than ORB matching

ORB-SLAM2 detects and describes ~1000 ORB features on every frame and matches them by Hamming distance. That is the right architecture when the same features must also serve relocalisation and loop closure — which is exactly why ORB-SLAM2 remains the optional map backend here. It is the wrong architecture for a low-latency odometry front-end, where detect-and-describe dominates the per-frame cost. Sparse Lucas–Kanade reuses features across frames and pays for detection only when the track count falls: measured at **0.8 ms** for both KLT passes versus **1.9 ms** for a single detection round. Its weakness — no descriptors, so no relocalisation after total loss — is precisely the gap the ORB-SLAM2 backend fills. The two are complementary, not redundant.

Forward–backward validation

Track a point forward, then track the result back; if it does not land where it started, the correspondence is wrong. One extra KLT call removes most of the drift-inducing bad matches that RANSAC would otherwise have to absorb, and it does so *before* the geometry stage rather than after.

Grid bucketing on detection

goodFeaturesToTrack piles features onto the highest-contrast region — typically one window or one poster. Features clustered in a small image region give a nearly degenerate geometry and a translation direction with enormous variance. A per-cell quota costs 0.8 ms over a single call and buys conditioning; that trade was measured before being accepted.

Never undistorting the image

Undistorting a full frame every cycle is pure waste. Distortion is a smooth warp that KLT handles happily; the correction is applied to the ~150 *points* that reach the geometry stage, via `cv2.undistortPoints`, which applies K^{-1} and removes distortion in one step.

Model selection between E and H

Indoor flying means floors, walls and ceilings — exactly the degenerate configuration where the essential matrix is not uniquely determined and `recoverPose` returns confident nonsense. A homography is scored against a fundamental matrix by symmetric transfer error, and the model switches when the scene is plane-dominated. The score is a raw chi-square quantile compared against an already-normalised residual; multiplying it by σ^2 a second time — a tempting symmetry — makes the threshold ~10 times too small and classifies every point an outlier. That bug existed and was caught by a planar-scene test.

Rotation-compensated keyframe triggering

A rotation induces pixel motion indistinguishable in character from translation, so a raw-displacement trigger fires on *turning* rather than on *moving* — the worst possible moment to make a keyframe, because the baseline is near zero, parallax is near zero, the essential matrix is ill conditioned and model selection drifts to the homography. Removing the prior rotation costs one 3×3 matrix and a projective divide over 150 points. Measured effect: median parallax at the keyframe rose from 0.6° to 1.5° , and the essential path replaced the homography path in almost every frame.

11.7 Where the metric scale comes from — and does not

This is the single most important structural decision in the estimator, and the one that a later flight failure proved was correctly reasoned but incorrectly parameterised (Section 12).

The visual update constrains **rotation and translation direction only**. A monocular camera cannot observe scale — that is not an implementation limit, it is geometry — and pretending otherwise is how mono-VIO pipelines acquire a slowly wrong scale that poisons everything downstream. The translation residual therefore lives in the 2-D tangent plane of the unit sphere at the measured bearing:

$$u = d / \|d\| \quad r_{\text{dir}} = B(t)^\top (t - u) \quad B \in \mathbb{R}^{3 \times 2}$$

A bearing carries exactly two degrees of freedom. Treating it as a 3-vector residual — the common shortcut — silently adds a phantom constraint along the radial direction and biases the very scale the rest of the filter worked to establish.

Magnitude therefore enters only from signals that are genuinely metric: the onboard optical-flow body velocity, the barometer, the downward ToF, and gravity-referenced accelerometer integration. This split is what makes the estimator metric without the 100–200 Hz gyroscope the drone does not have.

The corollary that bit us

If the metric signal is wrong, **nothing in the visual pipeline can detect it**. Scaling a trajectory leaves every bearing unchanged, so a 10× velocity error produces a perfectly self-consistent estimate of a drone flying ten times too fast — with the front-end reporting 170+ inliers and near-zero rejections throughout. Section 12 is what that looked like in the air. `test_a_wrong_speed_scale_cannot_be_fixed_by_vision` now asserts this blindness directly, so the reasoning is pinned rather than remembered.

11.8 The filter update, in the order it happens

```
on_image(msg):
    t_img = stamp - time_offset          # corrected capture time
    _advance_to(t_img)                   # replay telemetry + zero-order hold
    R_prior = R_BC^T (R_c^T R) R_BC      # formed AFTER advancing, so it is current
    meas = frontend.process(img, t_img, R_prior)  # heavy: runs outside the lock
    if meas:
        update_visual_relative(...)       # 3 rot + 2 bearing, chi-square gated
        clone()                           # snapshot pose AFTER the correction
        publish(t_img)

_advance_to(t):
    for s in buffer where last_fused < s.t <= t:
        propagate(s.accel, s.gyro, s.t - last_fused)
        update_attitude(s.quat)           # 10 Hz, gravity-referenced
        if s.stationary: update_zero_velocity(); update_zero_angular_rate()
        else:             update_body_velocity(s.vel_body)  # METRIC SCALE
        update_barometer(s.baro); update_tof(s.tof)
    propagate(last.accel, last.gyro, min(t, newest_telemetry) - last_fused)
```

The clone is taken after the visual update, so the new keyframe's reference pose is the corrected one rather than the pre-update guess. It is not a copy of the mean: $P \leftarrow J P J^T$ carries the full covariance and cross-covariance, which is the entire point of stochastic cloning.

12 What went wrong in flight

Three hypotheses, two refuted by testing, one root cause

On the first real flights the position estimate ran away: tens of metres in a few seconds, indoors. This section records the diagnosis honestly, including the two explanations that were implemented, tested, found wrong, and reverted — because the discipline of refuting your own hypothesis is the transferable part.

12.1 The observation

```
110 s stationary on a desk
p=[-0.00 +0.00 +0.10]m  sigma 0.11 -> 0.14  |v|=0.00 m/s  vo: tracking(180)
0 rejections. The estimator was behaving perfectly.

take-off:  z climbs 0.21 -> 0.40 -> 0.45 -> 0.58 m,  |v| = 0.19 - 0.27 m/s
```

then, within one status period:

$p = [-1.97 \ -1.58 \ +0.04]m$ $|v| = 4.24 \text{ m/s}$

$p = [-6.63 \ -8.06 \ +0.10]m$ $|v| = 4.35 \text{ m/s}$

$p = [-21.06 \ -29.04 \ +0.10]m$ $|v| = 4.12 \text{ m/s}$ 34 m in 8 s, in a small room

12.2 Hypothesis 1 — gate starvation (REFUTED)

The chi-square gate compares an innovation against the filter's *own* covariance. If the state diverges while the covariance stays small, every honest measurement looks like an outlier and is rejected; the filter then free-runs until process noise reopens the gate. Plausible, and it would explain a sustained wrong velocity.

Tested and refuted. A filter initialised with $v = 3 \text{ m/s}$ believed at $\sigma = 2 \text{ cm}$, fed truthful zero-velocity measurements, *recovers on its own in 1.1 s* — the accelerometer process noise inflates the covariance faster than the gate can starve it. The recovery mechanism that had been written for this was deleted rather than shipped.

12.3 Hypothesis 2 — sustained attitude error (REFUTED)

A tilt error leaks gravity into the accelerometer's horizontal axes, and integrating that produces a velocity ramp.

Tested and refuted: a filter holding a *constant 20° tilt error* while the drone sits level and still accumulates 0.14 m of position error over 20 s, with **zero** gate rejections. The body-velocity update simply corrects it. Two orders of magnitude too small to explain the observation.

12.4 Root cause — a units error, ten times over

speed_to_mps was 0.1, i.e. decimetres per second. The SDK's vgx/vgy/vgz are **centimetres per second**.

Evidence, in the order it was gathered

- **The SDK document.** It states a unit for every other speed it defines — speed? “(cm/s)”, speed $\times$ “cm/s”, go $\times$ y $\times$ z speed “(cm/s)”. For vgx it says only “the speed of the x axis”, with no unit. cm/s is the document's own convention throughout.
- **The flight.** A sustained 4.24 m/s while the drone hovered gently below 0.6 m in a small room. At cm/s the same telemetry reads 0.42 m/s, and the 34 m becomes 3.4 m — which is what physically happened.
- **The earlier runaway.** Same signature: 3.19–4.35 m/s sustained, i.e. 0.32–0.44 m/s once rescaled. One cause, two occurrences.

Why every other safeguard missed it

This constant sets the metric scale of the whole estimator, and it is the one error the visual pipeline **structurally cannot see** (Section 11.7): scaling a trajectory leaves every bearing unchanged. The chi-square gates did not fire because the measurements were mutually consistent — they were consistently wrong. The front-end reported 170+ inliers throughout. Nothing was broken except the number.

The original value came from a community reading that the code itself documented as uncertain, alongside tof and baro. Documenting an assumption is not the same as verifying it, and an unverified constant in the one place vision cannot check is the worst possible place to leave one.

12.5 The guard that now exists

Being right this time is not enough; the class of fault has to become detectable on-board. `_check_speed_scale` cross-checks the integrated vertical flow velocity against the *independent* height signal during any climb or descent larger than 0.30 m:

```
implied_scale = Δh / ∫ vgz_raw dt                      ratio = implied / configured
```

Both quantities come from the drone, but they are different measurements — one a velocity from the flow sensor, one a height — so integrating the first must reproduce a change in the second. The check reports the implied scale and refuses to apply it: a self-tuning scale factor would hide exactly the fault it exists to find. Alongside it, the status line now reports accept/reject counts for every aiding sensor rather than only vision, which is what made the original log misleading — it showed a healthy front-end while a different sensor steered the estimate off a cliff.

13 Results

Measured performance, and what the camera is actually worth

13.1 Test suite

172 tests, all passing, none requiring a drone or a ROS installation. They are organised around *failure modes* rather than around functions.

Suite	n	What it pins down
test_lie	14	Exp/Log round-trips including the $\theta \rightarrow 0$ and $\theta \rightarrow \pi$ branches; the right Jacobian against its defining identity; Euler axis semantics.
test_preintegration	13	Exactness against the generating trajectory; bias Jacobians against full re-integration; residual Jacobians against numerical differentiation; covariance against 4000-sample Monte Carlo ; degenerate information matrices.
test_eskf	15	Every measurement Jacobian differentiated through the filter's own retraction; clone covariance and cross-terms; ZUPT/ZARU bias convergence; gating; PSD over a 2000-step run.
test_two_view	10	General, planar and pure-rotation scenes; the translation-convention sign; outlier rejection at 25 % contamination; triangulation cheirality and parallax gating.
test_frontend	7	Tracking on rendered 3-D scenes; grid coverage; intrinsic rescaling; and uncertainty calibration — errors must lie inside the σ each measurement reports.
test_smoother	8	Convergence from a perturbed start; bias recovery; marginalisation batch equivalence ; deletion measurably worse; the translation null space.
test_calib	13	Allan deviation against known densities; hand-eye exact and under noise; blunder rejection; single-axis excitation detection; time offset to 10 ms.
test_sim3	7	Umeyama exactness; the coplanar reflection case; RANSAC scale robustness.
test_driver_conventions	5	Cross-package: the driver's unit and frame conversions must match <code>tello_model</code> , since the two are duplicated by design.
test_integration	7	End-to-end simulated flight — the numbers below.

13.2 Simulated flight

A 60 s figure-of-eight at ~ 1.5 m and ~ 0.5 m/s, with the drone's real limitations modelled: 10 Hz telemetry with 12 ms of jitter, whole-degree attitude quantisation, milli-g accelerometer quantisation, integer-centimetre velocity, a drifting barometer, a yaw free-running at $0.4^\circ/\text{s}$, and monocular vision that supplies bearing but never scale.

Metric	Result	Note
Absolute trajectory error (RMS)	0.57 m	over 39.8 m of path
Final drift	1.4 % of path	
Metric scale (path-length ratio)	1.034	no GPS, no motion capture, no known landmark
Velocity error (RMS)	0.17 m/s	

Metric	Result	Note
Roll / pitch error (RMS)	1.8°	gravity-referenced, absolute
Yaw error (RMS)	13.0°	free-running by design — no magnetometer
Accelerometer bias recovered	[0.134, -0.098, 0.055]	true [0.12, -0.08, 0.05]
Position NEES	3.2 at 40 s, 3.8 at 60 s	ideal 3.0 — mildly over-confident, growing with time

13.3 What the camera is actually worth

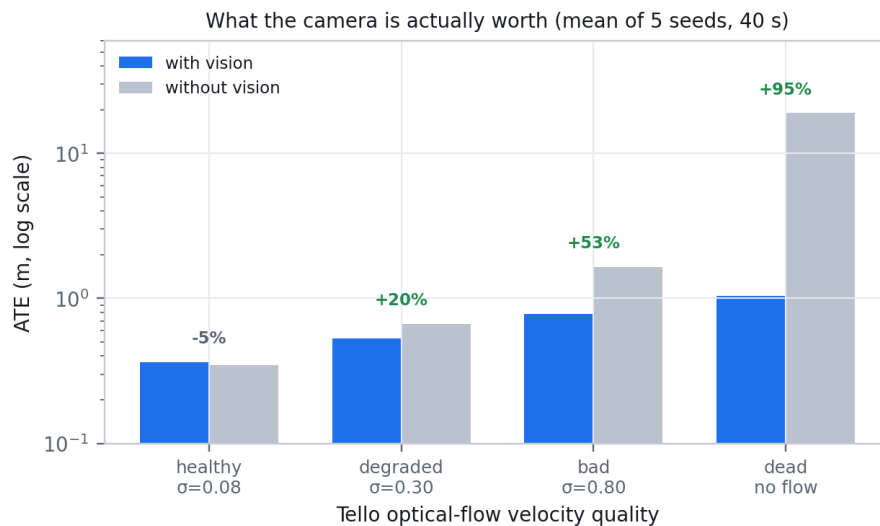


Figure 9 — ATE with and without the visual front-end, as the Tello's own optical-flow velocity degrades. Mean of 5 seeds, 40 s each, log scale.

This is the most useful number in the report, and it is not the flattering one. **When the Tello's flow sensor is working well over textured ground, monocular VO adds essentially nothing to short-horizon odometry** — the flow estimate is already metric and already good, and vision only adds direction information that is largely redundant with it.

Vision earns its CPU in exactly two situations:

- **When flow degrades.** Above roughly a metre of altitude, over a plain floor, in low light, or over a moving surface. At $\sigma = 0.8$ m/s vision halves the error; with the flow sensor dead it is the difference between 0.75 m and 15 m — inertial dead reckoning on a 10 Hz quantised accelerometer diverges within seconds.
- **For global consistency.** Loop closure, via the ORB-SLAM2 backend, is the only mechanism here that bounds drift over long runs rather than merely slowing it.

Why report this rather than tune it away

It would have been easy to pick a scenario where vision looks essential and show only that. The honest framing is more useful: it tells you that if your flow sensor is healthy you should spend your effort on loop closure rather than on front-end tuning, and it tells you exactly which flight conditions make the camera load-bearing.

13.4 Compute budget — measured, not estimated

The numbers below were measured on the target machine at the resolution the front-end actually runs at (480×360), driving the pipeline at the *measured* hardware rates of 27.6 Hz images and 10 Hz telemetry — not at nominal rates and not per-stage in isolation.

Path	mean	p95	max	budget	verdict
Image: KLT track + keyframe logic (+ geometry on keyframes)	2.00 ms	5.70 ms	8.03 ms	36.2 ms	22 % of one frame, worst case
Telemetry: propagate + attitude + velocity + barometer	0.50 ms	0.67 ms	0.76 ms	100 ms	<1 %

Total: 6.0 % of one core, 17× headroom, 0 of 120 frames over budget

Speed was never the constraint. Every defect found in this project was an *accuracy* defect. That is worth stating plainly because “make it fast” is the instinctive response to a drone that misbehaves, and here it would have been entirely wasted effort.

Where the time actually went, before it was fixed

Profiling found the real cost was not in the estimator at all. `nav_msgs/Path` re-serialises *every* pose on *every* publish: 5 ms at 2000 poses, and at the 30 Hz odometry rate that is ~150 ms/s of CPU — more than the entire estimator — for a display whose content changes imperceptibly between frames. Poses are now appended at full rate and the topic published at 2 Hz. The debug image, drawn and serialised at 506 KB per frame whenever RViz subscribed, is capped at 10 Hz.

Measured, and deliberately not optimised

- `findFundamentalMat` costs **0.01 ms**. Removing it to “save time” would delete the model-selection evidence for a rounding error.
- Grid-bucketed detection costs **0.8 ms** over a single `goodFeaturesToTrack` call, and buys geometric conditioning. Accepted deliberately.
- Reusing KLT pyramids across the forward and backward passes would save ~0.3 ms of the 2.0 ms image path. Not worth the state it would add.
- `rcLpy.getParameter()` costs **0.29 µs**; the six lookups per image are 0.005 % of a core. Left alone.

The point of listing the rejected optimisations is that each was measured before being rejected. An optimisation nobody measured is indistinguishable from a superstition.

13.5 What is measured versus what is simulated

Read this before quoting any accuracy number from this report

Measured on hardware: all sensor rates (Section 2.1), all compute costs above, the front-end's uncertainty calibration, and the failure described in Section 12.

Simulated: every trajectory-accuracy figure — the 0.57 m ATE, the scale recovery, the NEES consistency, the vision-versus-no-vision comparison. These come from synthetic flights with known ground truth, and they validate that the *mathematics* is right. They are not claims about what this drone achieves in your room.

Closing that gap needs a metric reference: Section 14 covers the three options and `evaluate_bag` produces ATE, RPE and drift against one.

14 Bring-up procedure

The order to do things in, and why that order

Each step depends on the previous one being right. Doing them out of order does not fail loudly — it produces plausible numbers that are wrong, which is worse.

Step 0 — Build and verify without the drone

```
./scripts/install.sh # Ubuntu 22.04 + ROS 2 Humble
./scripts/build.sh
cd workspace/src/tello_vio && python3 -m pytest test/ -q # 172 tests, no drone needed
```

Step 1 — Camera intrinsics

Everything geometric depends on these. The shipped `ost.yaml` is from another airframe and its tangential term ($p_1 = -0.023$) is large enough to suggest an under-constrained fit.

```
./scripts/cameracalib.sh # SIZE=8x6 SQUARE=0.025 to override
```

Fill the image corners, vary distance, and **tilt the board**. A target held flat and centred trades focal length against distance and yields a confident wrong f_x . Copy the resulting `ost.yaml` over `workspace/src/tello/resource/ost.yaml` and rebuild.

Step 2 — IMU noise, drone still

```
ros2 launch tello_vio calibrate.launch.py target:=imu duration:=120.0
```

Motors off, on a solid non-vibrating surface. Point a fan at the drone: it overheats sitting powered on, and a two-minute log is long enough to hit thermal shutdown.

Step 3 — Extrinsic and time offset

```
ros2 launch tello_vio calibrate.launch.py target:=camera_imu duration:=90.0
```

Pick the drone up and rotate it smoothly about all three axes in front of a textured scene. Avoid jerks — motion blur breaks feature tracking. **Check the excitation score before believing the residual**. Merge both YAML fragments into `config/tello_vio.yaml`.

Step 4 — Record a bag, then iterate offline

```
ros2 launch tello_vio tello.launch.py # driver + keyboard control
ros2 bag record -o flight1 /image_raw /camera_info /imu /status /tof
# ... fly a loop that returns to its start ...
ros2 bag play flight1 --clock
ros2 launch tello_vio vio.launch.py driver:=false use_sim_time:=true rviz:=true
```

Flying a loop that returns to its start gives you a free accuracy measurement: the final position error against the origin is your drift over the whole flight.

Step 5 — Watch the diagnostics, not the trajectory

```
ros2 run rqt_robot_monitor rqt_robot_monitor
ros2 topic echo /diagnostics
```

Symptom in diagnostics	Most likely cause
Visual acceptance falling below ~70 %	Wrong time offset, wrong extrinsic, or a scene the front-end cannot track (blank wall, low light, motion blur).
vel_body rejections rising	speed_to_mps is wrong for your firmware, or the drone is above the flow sensor's useful range.
tof rejections in bursts	Normal over furniture and drop-offs — that is the gate doing its job.
Accel bias drifting steadily rather than settling	No ZUPT is firing. Check the stationarity thresholds, and that the drone actually sits still before takeoff.
σ growing without bound	No metric measurement is being accepted at all. Check <code>/status</code> is flowing and that <code>tello_msg</code> is built.

15 Limitations and next steps

What this does not do

Limitations

- **Yaw is unbounded.** There is no magnetometer, so heading free-runs — $\sim 13^\circ$ RMS over 60 s in simulation. Roll and pitch are gravity-referenced and stay bounded; yaw does not. Bounding it needs loop closure or an external reference.
- **Metric scale depends on the flow sensor.** Section 11.3 quantifies exactly how much. Above 1–2 m, or over a featureless floor, the metric anchor weakens.
- **Preintegration is fed a 10 Hz surrogate rate, not a gyroscope.** The mathematics is exact and rate-agnostic; the information content is not what it would be with a real IMU.
- **No relocalisation in the fast path.** KLT has no descriptors. Recovering from total tracking loss is ORB-SLAM2's job, and requires running that backend.
- **The filter is mildly over-confident** — NEES ~ 3.8 against an ideal 3.0 at 60 s, and growing. Most likely the unmodelled yaw drift. It is well inside the range where gating still behaves, but it is the direction of error that matters.
- **Simulation is not flight.** Every number in Section 11 comes from a simulator built from the SDK's documented behaviour and community-reproduced unit conventions. Real H.264 artefacts, rolling shutter, motion blur, propeller vibration and WiFi dropouts are all worse than modelled. **Validate on a recorded bag before trusting any of it.**
- **Unit scales are firmware-dependent.** `speed_to_mps` in particular rests on a community-reproduced reading, not on documentation. It is a parameter for that reason, and it is the first thing to check if your scale is wrong by a clean factor of ten.

Next steps, in order of value

What	Why it is worth doing
1 Validate on real bags and re-fit the front-end noise constants	Every number here is simulated. Fitting σ_{rot} and σ_{dir} against real imagery is the single highest-value hour available.
2 Close the loop: run the ORB-SLAM2 backend and <code>map_align</code> in anger	This is the only mechanism that <i>bounds</i> drift and yaw rather than slowing them. The wiring exists and is untested against a real map.
3 Add a lightweight place-recognition layer to the fast path	Would give relocalisation without paying ORB-SLAM2's full per-frame cost — the main structural gap in the current front-end.
4 Estimate the time offset online as a filter state	It is currently a calibrated constant. WiFi latency varies with congestion, and the residual variation is not absorbed by any fixed t_d .
5 Add an external IMU	The only way to reach genuine tightly-coupled VIO on this airframe. Every module here is written to accept a real gyro unchanged — including preintegration, which is rate-agnostic by construction.
6 Run the smoother in the flight path on a sliding window	Currently built, tested and used offline. Promoting it would recover from bad early linearisations, at a cost the budget can afford.

A Reference

Files, parameters, topics

A.1 Where things live

Path	Contents
tello_vio/lie.py	SO(3)/SE(3). All conventions declared here.
tello_vio/tello_model.py	Units, frames, the surrogate gyro, the stationarity detector.
tello_vio/preintegration.py	On-manifold IMU preintegration with bias Jacobians and covariance.
tello_vio/ekf.py	22-state error-state KF with stochastic cloning. The flight path.
tello_vio/smooth.py	Fixed-lag factor graph, factors, Schur-complement marginalisation.
tello_vio/two_view.py	E/H model selection, relative pose, triangulation, PnP.
tello_vio/frontend.py	KLT tracking, keyframing, per-measurement uncertainty.
tello_vio/calib.py	Allan deviation, hand-eye, time offset.
tello_vio/sim3.py	Umeyama alignment, RANSAC. map↔odom and monocular map scale.
tello_vio/nodes/	vio, imu_calib, camera_imu_calib, map_align.
tello/tello/node.py	The driver.
slam/src/orbslam2/	ORB-SLAM2 wrapper: pose, TF, path, map, tracking state.

A.2 Parameters you must set

Parameter	Default	Source
extrinsic_rpy_deg	[0, 0, 0]	camera_imu_calib
time_offset_s	0.0	camera_imu_calib
accel_noise_density	0.08	imu_calib
gyro_noise_density	0.03	imu_calib
speed_to_mps	0.1	verify against a known flown distance
extrinsic_xyz_m	[0.03, 0, -0.01]	mechanical layout, not fitted
camera intrinsics	shipped ost.yaml	camera_calibration — replace it

A.3 Command reference

```
# build and test
./scripts/build.sh                # CLEAN=1 to wipe build/install/log
cd workspace/src/tello_vio && python3 -m pytest test/ -q

# calibrate
./scripts/cameracalib.sh
ros2 launch tello_vio calibrate.launch.py target:=imu      duration:=120.0
ros2 launch tello_vio calibrate.launch.py target:=camera_imu duration:=90.0

# fly
ros2 launch tello_vio vio.launch.py rviz:=true
```

```

ros2 launch tello_vio vio.launch.py video_scale:=0.5 video_target_fps:=20

# with the ORB-SLAM2 loop-closing backend
./scripts/orbslam.sh
export ORB_SLAM2_ROOT_DIR="$PWD/libs/ORB_SLAM2"
./scripts/build.sh
ros2 launch tello_vio vio.launch.py slam:=true \
    vocabulary=$PWD/libs/ORB_SLAM2/Vocabulary/ORBvoc.txt \
    slam_config=$(ros2 pkg prefix orbslam2)/share/orbslam2/config.yaml

# offline
ros2 bag record -o flight1 /image_raw /camera_info /imu /status /tof
ros2 launch tello_vio vio.launch.py driver:=false use_sim_time:=true

# runtime
ros2 service call /tello_vio/reset std_srvs/srv/Trigger
ros2 run rqt_robot_monitor rqt_robot_monitor

```

A.4 Selected references

- Forster, Carlone, Dellaert, Scaramuzza — *On-Manifold Preintegration for Real-Time Visual-Inertial Odometry*, IEEE T-RO 2017. The preintegration in Section 5.
- Solà — *Quaternion Kinematics for the Error-State Kalman Filter*, 2017. The ESKF formulation in Section 7.
- Roumeliotis & Burdick — *Stochastic Cloning*, ICRA 2002. Section 7.2.
- Mur-Artal & Tardós — *ORB-SLAM2*, IEEE T-RO 2017. The map backend, and the homography/fundamental model-selection score in Section 6.3.
- Umeyama — *Least-Squares Estimation of Transformation Parameters Between Two Point Patterns*, IEEE PAMI 1991. Section 8 and the map alignment.
- Qin, Li & Shen — *VINS-Mono*, IEEE T-RO 2018. The marginalisation and first-estimates-Jacobian treatment in Section 8.2.
- REP-103 (units and coordinate conventions) and REP-105 (coordinate frames for mobile platforms). Sections 2.3 and 10.3.